

Trabajo Práctico - Programación 1

Segunda Parte: Ampliación y mejora del sistema desarrollado

1. Introducción

En la primera parte del Trabajo Práctico se desarrolló un sistema informático destinado a resolver una problemática determinada, aplicando los conceptos fundamentales de programación vistos durante la cursada.

En esta segunda parte deberán continuar trabajando sobre el mismo proyecto desarrollado anteriormente, incorporando nuevas funcionalidades y mejorando la solución existente.

El objetivo de esta etapa es que el sistema evolucione hacia una aplicación más completa, robusta y organizada, incorporando los nuevos conceptos trabajados durante la cursada.

No se deberá realizar un proyecto completamente diferente. La segunda parte deberá partir del sistema presentado en la primera entrega y demostrar una evolución concreta del mismo.

2. Objetivos

El sistema deberá incorporar y demostrar la aplicación de los siguientes conceptos:

- Pruebas unitarias.
- Conjuntos (set).
- Tuplas (tuple).
- Excepciones.
- Diccionarios (dict).
- Archivos de texto.
- Archivos JSON.

Los conceptos deberán utilizarse de manera justificada y coherente con la problemática planteada.

No será suficiente con incorporar una estructura de datos o una funcionalidad únicamente para cumplir con el requisito. Cada elemento deberá tener una utilidad concreta dentro del sistema.

3. Continuidad de la primera parte

La segunda parte deberá construirse sobre el sistema desarrollado en la primera entrega.

Deberán:

- Mantener la problemática original.
- Mantener y mejorar las funcionalidades desarrolladas anteriormente.
- Corregir errores detectados en la primera versión.
- Incorporar nuevas funcionalidades.
- Reestructurar el código cuando resulte necesario.
- Incorporar los nuevos conceptos solicitados.

No se aceptará presentar un sistema completamente diferente al desarrollado en la primera parte.

La segunda entrega deberá demostrar una evolución del proyecto original.

4. Requisitos funcionales

El sistema deberá incorporar nuevas funcionalidades relacionadas con la problemática elegida.

Como mínimo deberá contemplar:

- Nuevas operaciones sobre los datos.
- Nuevas validaciones.
- Manejo de errores.

- Persistencia de información.
- Recuperación de información almacenada.
- Pruebas automáticas de funciones relevantes.

Las nuevas funcionalidades deberán estar integradas al sistema y no implementadas como ejemplos independientes.

5. Estructuras de datos

5.1. Conjuntos

El sistema deberá utilizar al menos un conjunto (set).

Deberá tener una finalidad concreta dentro de la aplicación, por ejemplo:

- Evitar elementos duplicados.
- Obtener valores únicos.
- Comparar información.
- Obtener elementos comunes o diferentes.

Deberán justificar su utilización en la documentación.

5.2. Tuplas

El sistema deberá utilizar al menos una tupla (tuple).

La tupla deberá utilizarse para representar información que tenga sentido mantener agrupada y que no requiera ser modificada durante su utilización.

Deberán explicar:

- Qué información representa.
- Por qué se decidió utilizar una tupla.
- En qué parte del sistema se utiliza.

5.3. Diccionarios

El sistema deberá incorporar diccionarios (dict) para representar o gestionar información.

Podrán utilizarse, entre otras posibilidades, para:

- Representar registros.
- Asociar identificadores con información.
- Administrar configuraciones.
- Contabilizar elementos.
- Organizar información para realizar búsquedas.

El uso de diccionarios deberá integrarse con las demás estructuras utilizadas.

6. Manejo de excepciones

El sistema deberá incorporar manejo de excepciones mediante try / ****except.

Las excepciones deberán utilizarse para controlar situaciones que puedan generar errores durante la ejecución.

Por ejemplo:

- Ingreso de datos con formato incorrecto.
- Conversión de tipos.
- Problemas al acceder a un archivo.
- Información inexistente o inválida.
- Operaciones que puedan generar un error.

No se deberá utilizar try / except indiscriminadamente para ocultar errores del programa.

Deberán identificar situaciones concretas en las cuales resulte necesario controlar una posible excepción y proporcionar un mensaje adecuado al usuario.

7. Archivos de texto

El sistema deberá utilizar archivos de texto para almacenar información.

Deberá implementarse como mínimo:

- Escritura de información.
- Lectura de información.
- Utilización posterior de la información almacenada.

La información deberá tener una utilidad concreta para el sistema.

8. Archivos JSON

El sistema deberá incorporar archivos JSON como mecanismo de almacenamiento de información.

Como mínimo deberá poder:

1. Generar o modificar información.
2. Guardar la información en un archivo .json.
3. Finalizar la ejecución del programa.
4. Volver a ejecutar el programa.
5. Recuperar la información almacenada.

Deberán utilizarse correctamente las operaciones necesarias para convertir la información entre las estructuras de Python y el formato JSON.

9. Pruebas unitarias

El proyecto deberá incorporar pruebas unitarias que permitan verificar el correcto funcionamiento de las funciones desarrolladas.

Cantidad mínima

Cada grupo deberá implementar un mínimo de 10 pruebas unitarias.

Las pruebas deberán estar distribuidas de la siguiente manera:

- 4 pruebas de casos válidos: deberán comprobar que las funciones producen el resultado esperado ante datos correctos.
- 3 pruebas de casos inválidos: deberán comprobar el comportamiento del sistema ante datos incorrectos o situaciones no permitidas.
- 2 pruebas de casos límite: deberán contemplar valores extremos o situaciones particulares que puedan afectar el funcionamiento de la función.
- 1 prueba adicional: deberá corresponder a un caso relevante para la problemática particular del sistema.

Distribución

Las 10 pruebas deberán corresponder a al menos 5 funciones diferentes del sistema.

No se considerará válido cumplir con el requisito realizando todas las pruebas sobre una única función.

Las pruebas deberán utilizar datos y situaciones diferentes y deberán verificar resultados concretos.

Requisitos adicionales

Cada prueba deberá permitir identificar claramente:

1. La función que se está probando.
2. Los datos utilizados.
3. El resultado esperado.
4. El resultado obtenido.
5. Si la prueba fue aprobada o no.

Las pruebas deberán poder ejecutarse de manera independiente del programa principal.

El código de las pruebas unitarias deberá formar parte de la entrega.

Durante la defensa del trabajo, el docente podrá seleccionar cualquiera de las pruebas presentadas y solicitar a los estudiantes que expliquen:

- Qué situación está comprobando.
- Por qué se eligió ese caso.
- Cuál es el resultado esperado.
- Qué ocurriría si se modificaran los datos utilizados.

resultado esperado.

10. Modularización

Se deberá mantener y profundizar la modularización implementada en la primera parte.

El programa deberá estar organizado en funciones con responsabilidades claras.

Se deberá evitar:

- Concentrar toda la lógica en el programa principal.
- Repetir código innecesariamente.
- Crear funciones que realicen demasiadas tareas diferentes.
- Utilizar variables globales para compartir información cuando pueda resolverse mediante parámetros y valores de retorno.

El programa principal deberá encargarse principalmente de coordinar el funcionamiento del sistema.

11. Validaciones

Las entradas proporcionadas por el usuario deberán ser correctamente validadas.

El sistema deberá contemplar situaciones como:

- Datos obligatorios sin completar.
- Valores fuera de rango.
- Opciones inexistentes.
- Tipos de datos incorrectos.

- Información que no corresponda con el estado actual del sistema.

Ante un dato incorrecto, el sistema deberá informar claramente el problema y permitir un nuevo ingreso cuando corresponda.

12. Restricciones técnicas

IMPORTANTE

El objetivo de este Trabajo Práctico es evaluar los contenidos y herramientas trabajados durante la materia.

Por este motivo, deberán respetarse las siguientes restricciones.

12.1. No se permite utilizar clases

No se permite la utilización de Programación Orientada a Objetos mediante clases.

No deberán utilizarse:

- class
- Objetos creados mediante clases propias.
- Herencia.
- Métodos de clases.
- Constructores.
- Otros mecanismos de Programación Orientada a Objetos que no hayan sido trabajados en la materia.

El proyecto deberá resolverse utilizando los conceptos de programación enseñados durante la cursada.

12.2. No se permite utilizar bases de datos

No se permite utilizar ningún tipo de base de datos.

Por lo tanto, no deberán utilizarse:

- MySQL.

- PostgreSQL.
- SQLite.
- SQL Server.
- MongoDB.
- Bases de datos en la nube.
- ORM.
- Sistemas de persistencia basados en bases de datos.

La persistencia de información deberá realizarse exclusivamente mediante los mecanismos de archivos trabajados en la materia, incluyendo archivos de texto y archivos JSON.

12.3. No se permite utilizar librerías no vistas en la materia

No se podrán incorporar librerías, módulos o herramientas externas que no hayan sido trabajados y autorizados durante la cursada.

Esto incluye librerías utilizadas para:

- Procesamiento de datos.
- Bases de datos.
- Interfaces gráficas.
- Conexiones a Internet.
- Consumo de APIs.
- Automatización.
- Inteligencia artificial.
- Procesamiento avanzado de archivos.
- Visualización de datos.
- Frameworks.

- O cualquier otra funcionalidad que no forme parte de los contenidos trabajados.

El hecho de que una librería sea estándar de Python no significa automáticamente que pueda utilizarse.

Únicamente podrán utilizarse las librerías, módulos y herramientas que hayan sido vistos y autorizados durante la materia.

12.4. Código externo

No se permitirá incorporar código obtenido de Internet, repositorios, herramientas de inteligencia artificial, otros proyectos o cualquier otra fuente externa sin comprenderlo y poder explicarlo.

Todo el código presentado deberá poder ser explicado por los integrantes del grupo durante la defensa del trabajo.

El docente podrá solicitar modificaciones sobre el código durante la presentación para comprobar la comprensión de la solución desarrollada.

13. Integración de los contenidos

Los conceptos solicitados deberán estar integrados dentro del sistema.

No se aceptará una implementación en la cual cada concepto aparezca aislado únicamente para cumplir con el enunciado.

Se espera que los estudiantes puedan resolver situaciones reales de su sistema utilizando adecuadamente las distintas herramientas.

Por ejemplo, una funcionalidad podría involucrar:

Diccionarios → Validación → Excepciones → Archivo JSON → Recuperación de información → Prueba unitaria

La integración podrá ser diferente según la problemática elegida.

Lo importante será demostrar que los estudiantes comprenden cuándo, cómo y por qué utilizar cada herramienta.

14. Requisitos mínimos obligatorios

Requisito	Obligatorio
Continuar el proyecto de la Parte 1	Sí
Mejorar funcionalidades existentes	Sí
Incorporar nuevas funcionalidades	Sí
Conjuntos	Sí
Tuplas	Sí
Diccionarios	Sí
Excepciones	Sí
Archivos de texto	Sí
Archivos JSON	Sí
Pruebas unitarias	Sí
Validaciones	Sí
Modularización	Sí
Clases	NO PERMITIDO
Bases de datos	NO PERMITIDO
Librerías no vistas en la materia	NO PERMITIDO

15. Documentación

Junto con el código deberán entregar un documento explicativo que incluya:

a. Descripción de las mejoras

Explicar qué funcionalidades fueron modificadas o incorporadas respecto de la primera parte.

b. Estructuras de datos

Explicar dónde se utilizaron:

- Conjuntos.
- Tuplas.
- Diccionarios.

Y justificar la elección de cada una.

c. Persistencia

Explicar:

- Qué información se almacena.
- En qué archivo.
- Qué información se almacena en formato de texto.
- Qué información se almacena en formato JSON.
- Cómo se recupera posteriormente.

d. Excepciones

Indicar qué situaciones pueden generar errores y cómo fueron controladas.

e. Pruebas unitarias

Indicar qué funciones fueron testeadas y qué casos fueron considerados.

f. Cambios respecto de la Parte 1

Identificar claramente qué aspectos corresponden a la primera versión y cuáles fueron incorporados o modificados en esta segunda etapa.

16. Entrega

La entrega deberá contener:

1. Código fuente completo del proyecto.
2. Archivos necesarios para ejecutar el sistema.
3. Archivos de texto utilizados por el programa.

4. Archivos JSON utilizados por el programa.
5. Código correspondiente a las pruebas unitarias.
6. Documentación del proyecto.

El proyecto deberá poder ejecutarse correctamente en otro equipo siguiendo las instrucciones proporcionadas por los estudiantes.

17. Presentación y defensa

Durante la presentación, cada grupo deberá poder explicar el funcionamiento de su sistema y justificar las decisiones tomadas.

El docente podrá solicitar que los estudiantes:

- Expliquen una función determinada.
- Modifiquen una parte del código.
- Justifiquen la utilización de una estructura de datos.
- Expliquen el funcionamiento de una excepción.
- Expliquen cómo se almacena y recupera información.
- Ejecuten una prueba unitaria.
- Detecten y corrijan un error en el código.
- Incorporen una pequeña modificación solicitada durante la defensa.

Todos los integrantes del grupo deberán conocer el funcionamiento del proyecto y poder explicar el código desarrollado.

18. Criterios de evaluación

Se tendrá en cuenta:

- Correcta implementación de los requisitos.
- Aplicación adecuada de los conceptos vistos en clase.
- Modularización y organización del código.

- Correcto manejo de excepciones.
- Uso adecuado de conjuntos, tuplas y diccionarios.
- Correcta lectura y escritura de archivos.
- Correcta utilización del formato JSON.
- Calidad y variedad de las pruebas unitarias.
- Validación de datos.
- Integración de los nuevos contenidos con el sistema desarrollado en la primera parte.
- Cumplimiento de las restricciones técnicas.
- Claridad de la documentación.
- Capacidad de los integrantes para explicar y defender el código.

La incorporación de una funcionalidad únicamente para cumplir formalmente con un requisito no será considerada suficiente.

Se evaluará especialmente la comprensión de los conceptos y su correcta aplicación dentro de una solución coherente.

Fecha de Entrega: 05/11